Test Execution

Prepared for:

Southern Alberta Institute of Technology (SAIT) Software

CPRG-305-C Software Testing and Deployment (Fall 2024)

Due Date: November 7, 2024

Set Up a Test Environment

Objective:

We're planning to create a test environment that directly look like production, where the Property Pro real estate website will run or should be functional. This setup allows you to monitor how the system works in a controlled, reasonable environment without concerning live data.

Steps:

1. Set Up Next.js: Copy the production setup in our development environment. We have to make sure the routing, server settings, and environment variables match production.
2. Configure Firebase: We use a separate Firebase project particularly for testing. Identical the configurations, containing authentication methods by using Google, GitHub to sign in.
3. Set Up MongoDB: We use a testing database that look like the production database structure. Fill it with sample data, such as:
    a. Sample user accounts (admin or manager, basic user roles like buyers and sellers)
    b. Sample property listings with realistic details (names, addresses, pricing, number of bedrooms, number of bathrooms etc.)
4. Check Dependencies: We should make sure all third-party libraries and APIs used in production are also offered in the test environment.
5. Isolate Testing Data: Confirm the database and user account data are separate from production. This will prevent accidental data cross-over.

Testing Phases

Functional Testing

Objective: Our objective is to prove that all features work as required and meet the stated business requirements.

Techniques:

1. Boundary Testing: Test at the boundary between valid and invalid inputs. For instance:
    - Price input range for properties (e.g., $1,000,000 - $2,000,000)
    - Number of bedrooms and bathrooms with minimum and maximum values.
    - Searching by specific location for instance by using city name or province name or by using both city and province name together at once to search the specific property
2. Equivalence Partitioning: Group inputs into classes and test representative values from each class. This helps cover a broad range of inputs efficiently.

- Example: Input fields for property descriptions can be categorized by length—empty, short, medium, and long.

3. Positive and Negative Testing: Test correct and incorrect scenarios:
   - Positive Tests: Test with valid inputs (e.g., valid login, correct search query format).
   - Negative Tests: Try invalid inputs (e.g., incorrect login credentials, invalid characters in search).

➢ **Execution**: Focus on key functions like:
   - CRUD Operations: Create, Read, Update, and Delete functions for property listings.
   - Search and Filter: Test search filters to ensure correct results based on criteria like price, location, and bedrooms.
   - Role-Based Access Control (RBAC): Ensure users with different roles (e.g., admin vs. guest) have appropriate access to pages and actions.

➢ **Outcome:** Document each test case in a Defect Tracking Spreadsheet, marking them as pass/fail, and noting any errors encountered.

Performance Testing

Objective:

- To assess how well the application performs under various loads, identifying any issues in speed, stability, and capacity.

Execution:

1. Simulate Load: Use tools like JMeter or Locust to mimic different levels of user traffic. Gradually increase the load to identify at what point the system becomes unstable.
2. Track Key Metrics:
   - Response Time: How quickly the app responds to requests.
   - Throughput: The number of requests handled per second.
   - Error Rate: Frequency of errors under load, helping to identify weak points.
3. Stress Test: Test the system beyond its normal load to understand its breaking point.

➢ Outcome: Document performance metrics in the Defect Tracking Spreadsheet for each load level, noting any areas where performance degrades.

Automated Testing (Optional)

Objective:

- To reduce repetitive work and ensure consistent results across multiple test runs.

Execution:

1. Set Up a Testing Tool: Use Cypress or Selenium for UI automation. These tools can automate user interactions like login, form submissions, and page navigation.
2. Create Test Scripts: Write automated scripts for repetitive tests, such as:
   - Login Tests: Test login functionality for different roles.
   - Property Listing Tests: Automate adding, viewing, updating, and deleting properties.
   - Search Filter Tests: Automate testing for different filter combinations.
3. Run and Maintain: Execute tests regularly to catch any new bugs after code changes, and update scripts as the application evolves.

➢ Outcome: Maintain a report for each test run, recording pass/fail results and screenshots if necessary for documentation.

Document Testing Outcomes

Objective:

- To track and summarize each day's testing activities for an organized review.

Date	Test Case ID	Tester	Test Description	Outcome	Notes
2024-11-05	TC001	Asenai	Login/logout functionality with Clerk	Pass	Clerk login/logout successfully implemented
2024-11-05	TC002	Robel	Property click navigates to details page	Pass	It precisely displaying specific property info
2024-11-05	TC003	Miebaka	Adding property to favorites	Pass	Favorites successfully saved to user profile
2024-11-06	TC004	Jerome	Manager can delete property	Pass	Manager role-based permissions validated
2024-11-06	TC005	Robel	Manager can edit property	Fail	It doesn't go to specific property to update
2024-11-07	TC006	Asenai	All new attributes display (e.g., build date)	Fail	Missing attributes on some properties
2024-11-07	TC007	Miebaka	Calculation is accurate	Pass	correct calculations displayed

Defect Tracking

Group 3 - Defect Tracker

DEFECT / ISSUE TRACKER

Status	Count		User Impact	Business Impact
1-New	6	1-Critical	6	3
2-Open	5	2-High	2	7
3-Closed	4	3-Medium	7	1
TOTALS	15	4-Low	0	4
			15	15

Standardizing Code Identification for Defect Corrections: A Strategy Using Defect Tracking

1. The Defect ID is use to identitfy relevant code changes.
This reference can be used in Github as a part of commit messages. eg. "Fix D001: Transition login from firebase to clerk
2. Document the Commit Hash for each defect
3. Create a specific branch for each defect named as the defect ID

Defect ID	Detect Date	Statues	user Impact	Business Impact	Subject	Issue	Internal Update	Commit Hash #
D001	11/5/2024	New	Critical	High	Login Change	We change our login in feature or function from firebase to Clerk		
D002	11/5/2024	Open	High	Medium	Specific propery display error	Property details page fails to load specific info	Debugging display logic for selected properties	
D003	11/5/2024	Open	Medium	Medium	Favorites Not Saving	Favorite property not saving properly for some users	Reviewing save logic in the favorite feature	
D004	11/5/2024	New	Medium	Medium	Mortgage Calculator	Incorrect calculations in mortgage calculator	Fixing calculation formulas	
D005	11/5/2024	New	Medium	Low	Missing Property Attributes	New attributes (e.g., build date) not displaying in UI	Verifying schema alignment with UI components	
D006	11/5/2024	New	High	High	Authentication Transition Bug	Session timeouts occurring after Firebase to Clerk switch	Adjusting session handling in Clerk settings	
D008	11/6/2024	Open	Medium	Low	Favorites List UI Glitch	Favorite properties UI flickering on save	Reviewing state management in frontend for favorites	
D009	11/6/2024	Closed	Low	Low	Property Listing Edit Issue	Edited property details not saving properly	Corrected issue with form submission and data update	
D010	11/6/2024	Open	High	Medium	Mortgage Calculator UI Lag	Delay in response when adjusting values in calculator	Optimizing calculation triggers and UI response	
D011	11/6/2024	Closed	Medium	Low	New Property Attributes Missing	Recently added attributes not appearing on some listings	Updated database schema to include all attributes	
D012	11/7/2024							

Test Case Execution Progress Tracking

- The objective of this outline is to track the attempted test cases alongside the` unsuccessful test cases regular.

Date	Total Test Cases	Attempted Test Cases	Unsuccessful Test Cases	Success Rate (%)
10/06-10/12/2024	40	10	4	60%
10/13-10/17	20	10	3	70%
10/20-10/26	20	15	5	66.7%
10/27-11/03	20	18	6	66.6%
11/04-11/09	25	15	4	73.3%

Success Rate Calculation:

Success Rate= ((Attempted Test Cases − Unsuccessful Test Cases) / Attempted Test Cases) × 100

This tracking helps identify how defect-free the software is and where additional testing efforts are required